Software Architecture and Design Specification

Project: Bank Management System

Version: 1.0

Authors: BMS_PES – Vaibhava, Ronit, Vijay, Tarini

Date: 12-10-2026

Status: Draft

Revision History

Version | Date | Author | Change Summary

1.0 | 12-10-2026 | BMS_PES | First draft

Approvals

Role | Name | Signature/Date

Author (SAD) | Ronit |

Reviewer | Vijay |

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Bank Management System (BMS), a web application for managing bank accounts and money movement within a single bank.

1.2 Scope

Covers the services for the Customer, Teller and Admin roles: authentication and role-based access, account opening and maintenance (including creation of the customer's login), deposits, withdrawals, fund transfers, transaction history and statements, staff management, audit logging, daily reports and simulated notifications. Out of scope: loans, cards, cheques, external payment networks (NEFT/UPI/RTGS), mobile app, real SMS/e-mail gateways, customer self-registration.

1.3 Audience

Developers, QA engineers, security reviewers, instructors and maintainers.

1.4 Definitions

BMS, RBAC, REST, JSON, JWT, ACID, DAO, STRIDE, TLS, ADR, FR, NFR, SR, RTM.

2. Document Overview

2.1 How to use this document
Provides the architecture (goals, components, pattern, technology, risks, traceability, security) and the design (sequence diagrams, API, error handling, UX) of BMS.

2.2 Related Documents
SRS v1.0 (requirement IDs BMS-F-/NF-/SR-), Software Test Plan v1.0 (test cases TC-..., RTM).

3. Architecture

3.1 Goals & Constraints

Goals: secure handling of customer data and money movements; consistent balances (all-or-nothing transactions); responsiveness (target: 95 % of transactions within 3 s with 50 concurrent users, BMS-NF-001); availability target 99.5 % (BMS-NF-002); modularity so that four developers can work in parallel; testability per layer.

Constraints: single-bank scope with no external payment networks; TLS 1.2+ for client–server traffic (BMS-SR-001); fixed stack (Python 3, FastAPI, SQLite via SQLAlchemy, HTML/CSS/JavaScript, pytest, GitHub Actions); delivery in two one-week sprints (19–23 Oct and 26–30 Oct 2026).

3.2 Stakeholders & Concerns

Customer: Security of funds and data, correct balances, ease of use
Teller: Fast account opening, deposits, withdrawals and search
Admin/Branch Manager: Control of staff access, audit trails, reports
Bank/Regulator: Traceability, audit retention, data protection
Developers & testers: Modularity, clear interfaces, testability

3.3 Component (UML) Diagram

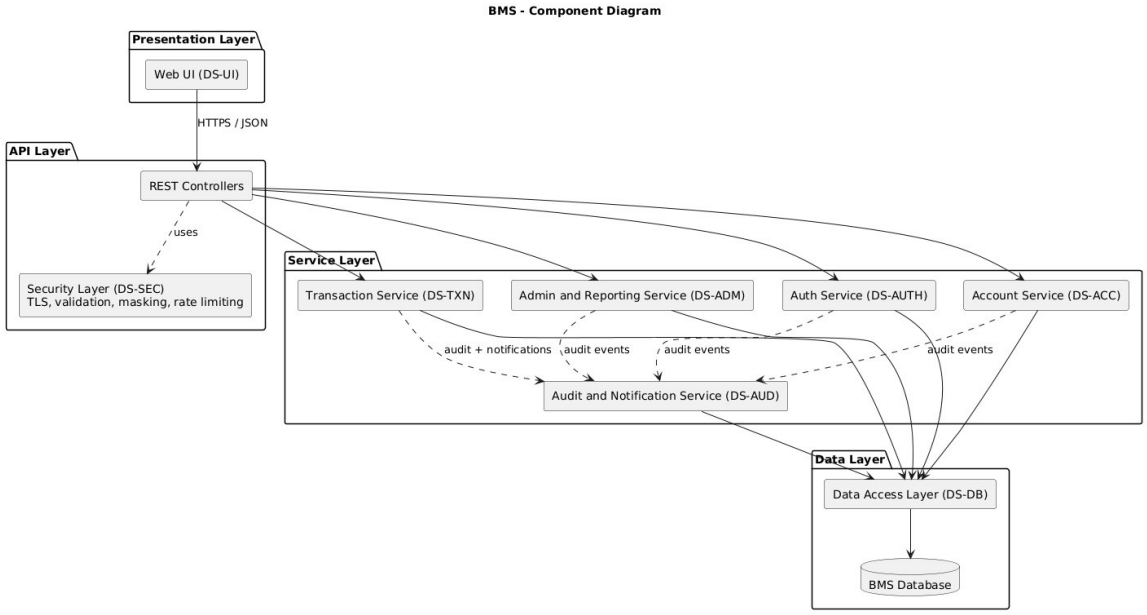


Figure 1 — Component diagram

The system is organised in layers. The Web UI calls REST controllers over HTTPS; controllers pass through the Security Layer and delegate to five services; services use a single Data Access Layer; every service reports security-relevant events to the Audit & Notification Service.

3.4 Component Descriptions

- Web UI (DS-UI): handles user input/output for Customer, Teller and Admin screens; performs client-side validation.

- Auth Service (DS-AUTH): validates login, locks accounts after 3 failed attempts, manages sessions and roles, handles password change.

- Account Service (DS-ACC): opens and maintains accounts, creates the customer login, freezes/closes accounts, balance view and search.

- Transaction Service (DS-TXN): processes deposits, withdrawals, transfers, history and statements as all-or-nothing transactions.

- Admin and Reporting Service (DS-ADM): manages Teller users and produces the daily transaction report.

- Audit and Notification Service (DS-AUD): records the append-only audit log and sends simulated transaction notifications.

- Security Layer (DS-SEC): enforces TLS, input validation, masking of account numbers and rate limiting.

- Data Access Layer and Database (DS-DB): account, customer, user, transaction, audit and notification data storage.

3.5 Chosen Architecture Pattern and Rationale

Layered architecture (Presentation, API, Service, Data Access, Database) chosen for clear separation of concerns, easier testing of each layer and parallel development by the team. Microservices rejected as overly complex for a single-bank system built by four developers in two one-week sprints.

3.6 Technology Stack & Data Stores

Python 3, FastAPI REST APIs, SQLite DB (via SQLAlchemy), HTML/CSS/JavaScript web UI, TLS encryption, bcrypt password hashing, pytest, GitHub Actions.

3.7 Risks & Mitigations

Risk: Partial update of balances during a failed transfer → Mitigation: every money movement runs in a single database transaction with rollback.

Risk: Unauthorised access to other customers' data → Mitigation: server-side role and ownership checks on every request.

Risk: Password guessing → Mitigation: lockout after 3 failed attempts and hashed passwords.

Risk: Slow response under 50 concurrent users → Mitigation: short transactions, indexing and a planned load test.

Risk: Tight two-sprint timeline → Mitigation: modules assigned to separate members using a shared requirement baseline.

3.8 Traceability to Requirements

BMS-F-001 (Login) → Auth Service (DS-AUTH)

BMS-F-002 (Account lockout) → Auth Service, Audit Service (DS-AUTH, DS-AUD)

BMS-F-003 (Role-based access) → Auth Service (DS-AUTH)

BMS-F-004 (Change password) → Auth Service (DS-AUTH)

BMS-F-010 (Open account) → Account Service, Auth Service, Transaction Service, Data Access Layer (DS-ACC, DS-AUTH, DS-TXN, DS-DB)

BMS-F-011 (Update customer details) → Account Service, Audit Service (DS-ACC, DS-AUD)
BMS-F-012 (Freeze/close account) → Account Service, Transaction Service, Audit Service (DS-ACC, DS-TXN, DS-AUD)

BMS-F-013 (View balance) → Account Service, Auth Service, Data Access Layer (DS-ACC, DS-AUTH, DS-DB)

BMS-F-014 (Search customer) → Account Service, Data Access Layer (DS-ACC, DS-DB)

BMS-F-020 (Deposit) → Transaction Service (DS-TXN)

BMS-F-021 (Withdrawal) → Transaction Service (DS-TXN)

BMS-F-022 (Fund transfer) → Transaction Service (DS-TXN, DS-DB)

BMS-F-023 (Transaction history) → Transaction Service (DS-TXN)

BMS-F-024 (Statement download) → Transaction Service (DS-TXN)

BMS-F-030 (Manage staff users) → Admin and Reporting Service (DS-ADM)

BMS-F-031 (View audit log) → Audit and Notification Service (DS-AUD)

BMS-F-032 (Daily report) → Admin and Reporting Service (DS-ADM)

BMS-F-033 (Notifications) → Audit and Notification Service (DS-AUD)

BMS-NF-001 (Performance) → Transaction Service (DS-TXN)

BMS-NF-002 (Availability) → Web UI / platform health endpoint (DS-UI)

BMS-NF-003 (Atomic transactions) → Transaction Service, Data Access Layer (DS-TXN, DS-DB)

BMS-NF-004 (Usability) → Web UI (DS-UI)

BMS-NF-005 (Audit retention) → Audit and Notification Service (DS-AUD)

BMS-NF-006 (Browser compatibility) → Web UI (DS-UI)

BMS-SR-001 (TLS 1.2+) → Security Layer (DS-SEC)

BMS-SR-002 (Password hashing) → Auth Service (DS-AUTH)

BMS-SR-003 (Session timeout) → Auth Service (DS-AUTH)

BMS-SR-004 (Server-side authorisation) → Auth Service (DS-AUTH)

BMS-SR-005 (Input validation) → Security Layer (DS-SEC)

BMS-SR-006 (Audit logging) → Audit and Notification Service (DS-AUD)

BMS-SR-007 (Masking and encryption of ID proofs) → Security Layer, Data Access Layer (DS-SEC, DS-DB)

3.9 Security Architecture

Threat Modeling (STRIDE): Spoofing → login with hashed passwords, lockout after 3 failed attempts and session timeout; Tampering → parameterised queries, input validation and all-or-nothing database transactions; Repudiation → append-only audit log of logins, financial and admin actions; Information disclosure → TLS 1.2+, masked account numbers, encrypted ID-proof numbers and no passwords in logs; DoS → rate limiting and health monitoring; Elevation of Privilege → server-side role-based access control (RBAC) on every request.

4. Design

4.1 Design Overview

BMS is designed with layered modules (API controllers, services, data access) for separation of concerns and easier maintenance. Business rules are implemented in the service layer, and services access the database only through the data access layer.

4.2 UML Sequence Diagrams

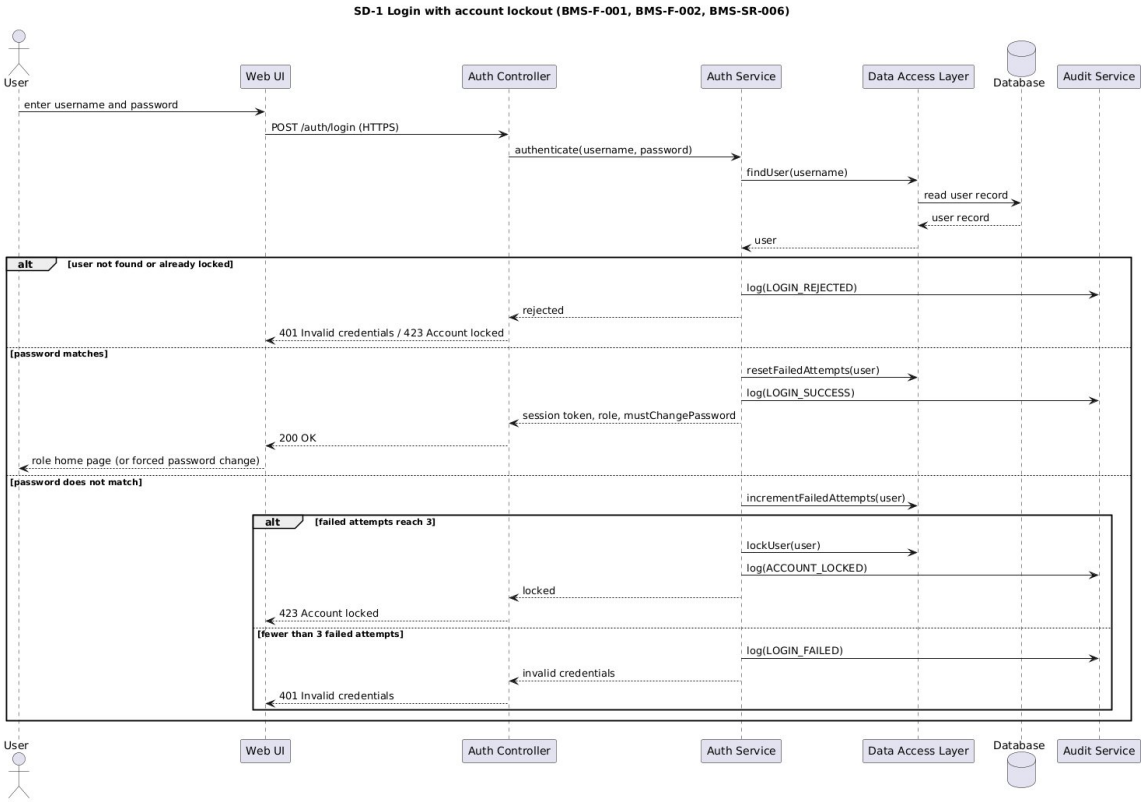


Figure 2 - Sequence diagram: Login with account lockout

A user submits credentials; the Auth Service checks them, resets or increments the failed-attempt counter, locks the account on the third failure and records an audit event.

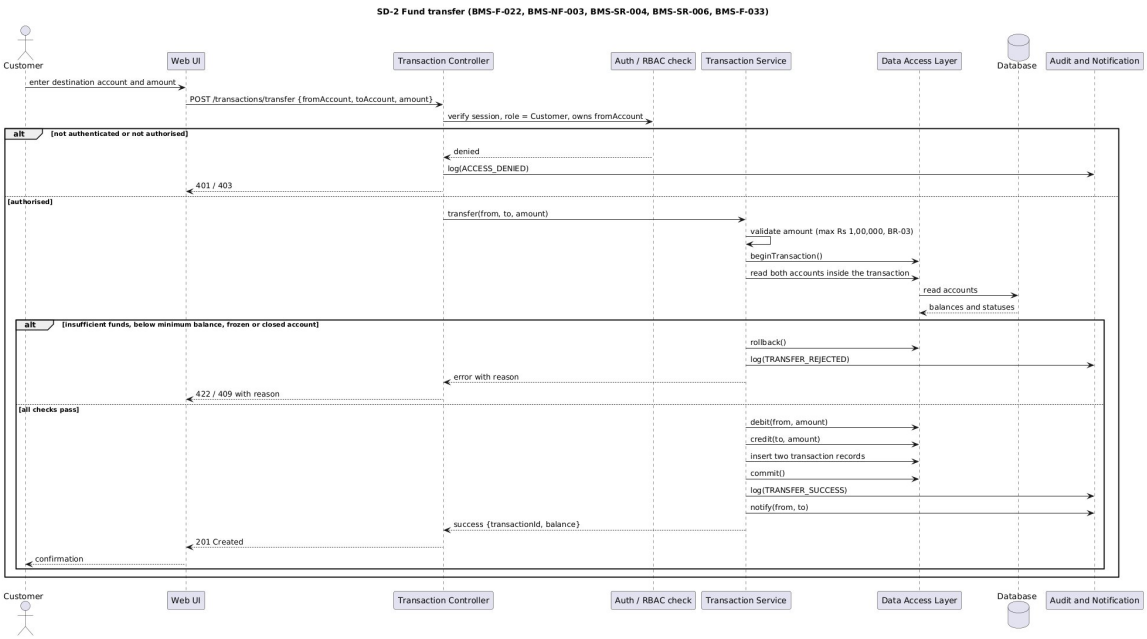


Figure 3 - Sequence diagram: Fund transfer

A Customer submits a transfer; after authorisation checks the Transaction Service debits and credits both accounts inside one database transaction, commits or rolls back together, then logs the event and sends a notification.

4.3 API Design

Endpoint: /auth/login

Method: POST

Request: {username, password}

Response: {token, role, mustChangePassword}

Errors: 401 Invalid credentials, 423 Account locked.

Endpoint: /auth/logout

Method: POST

Request: session token in header

Response: {status}

Errors: 401 Invalid or expired session.

Endpoint: /auth/password

Method: PUT

Request: {currentPassword, newPassword}

Response: {status}

Errors: 400 Weak password, 401 Wrong current password.

Endpoint: /transactions/deposit

Method: POST

Request: {accountId, amount}

Response: {transactionId, receiptNo, balance}

Errors: 400 Invalid amount, 403 Forbidden, 409 Account frozen or closed.

Endpoint: /transactions/withdraw

Method: POST

Request: {accountId, amount}

Response: {transactionId, balance}

Errors: 403 Forbidden, 409 Account frozen or closed, 422 Insufficient funds or daily limit exceeded.

Endpoint: /transactions/transfer

Method: POST

Request: {fromAccount, toAccount, amount}

Response: {status, transactionId, balance}

Errors: 403 Forbidden, 409 Account frozen or closed, 422 Insufficient funds or transfer limit exceeded.

4.4 Error Handling, Logging & Monitoring

Standardized error messages in one JSON format {code, message, timestamp, traceId}. No sensitive info in logs (no passwords, account numbers masked). Monitoring includes transaction failure rate, login failure count and system uptime through a /health endpoint.

4.5 UX Design

Web UI with accessible font sizes, clear error messages, a responsive layout that works down to 360 px, and a confirmation step before money is moved. Separate screens for Customer, Teller and Admin.

4.6 Open Issues & Next Steps

Future enhancements: multi-factor authentication, mobile app, real e-mail/SMS notification gateway, migration to PostgreSQL or MySQL.

5. Appendices

5.1 Glossary: BMS, RBAC, REST, JWT, ACID, DAO, STRIDE, TLS, ADR.

5.2 References: IEEE 42010, OWASP, NIST SP 800-160.

5.3 Tools: PlantUML, draw.io, Postman.